

A **Major Project Proposal** Report on

Project Title

Submitted in Partial Fulfillment of the Requirements for
the Degree of **Bachelor of Engineering in Computer Engineering**
under Pokhara University

Submitted by:

Aaditya Yadav	231201
Apil Khadka	231210
Ujashna Dangol	231247

Under the supervision of

[Supervisor Name]

[Designation]

Date: 04 Jun 2026

Department of **Computer Engineering**

**NEPAL COLLEGE OF
INFORMATION TECHNOLOGY**

Balkumari, Lalitpur, Nepal.

Abstract

Flint Secure is a real-time fraud detection and device intelligence platform designed for Nepal's digital payment ecosystem. The platform addresses financial fraud in emerging-market fintech, where transaction volumes are rising rapidly but prevention infrastructure remains immature. Nepal's FATF grey-list status and the documented 70% increase in SIM swap attacks (FIU-Nepal, 2024) underline the need for locally calibrated detection.

The system exposes a device identification API (three-layer fingerprint matching cascade) and a risk scoring API across seven signal dimensions: device trust, transaction amount, temporal behavior, velocity, network origin, identity consistency, and recipient risk. It returns a composite score (0–100) and an ALLOW, FLAG, or BLOCK decision within a sub-50 ms P95 latency target at approximately 3,200 requests per second.

The backend uses Go, dual Redis instances, NATS JetStream, and PostgreSQL, with Android, iOS, and JavaScript SDKs and a Python worker pipeline for trust scoring, behavioral profiling, and quota management. Validation is planned via a 168-test suite targeting $\geq 92\%$ pass rate.

Keywords:

1 Problem Statement

Nepal's digital payment landscape has grown rapidly. Mobile wallets (eSewa, Khalti, IME Digital) now serve millions of users, yet most payment service providers (PSPs) still rely on rule-based fraud checks—simple amount or frequency thresholds that cannot adapt to evolving attacks such as SIM swap, credential stuffing, device emulation, and money mule networks.

The fundamental problem is the absence of real-time, device-level fraud intelligence in Nepal's fintech stack. Current systems share three critical limitations:

- Rule-based systems cannot adapt without manual reconfiguration.
- No device identity layer exists—transactions cannot be tied to physical devices, blocking detection of one-device-many-accounts patterns.
- Behavioral profiling is absent—there is no per-user baseline for anomalous amounts, times, or recipients.

The consequence is high false negative rates (fraud undetected) and, where systems are overly conservative, high false positive rates (legitimate users blocked), harming both security and revenue.

2 Project Objectives

The primary objective is to design, implement, and validate a production-grade fraud detection platform for Nepal's digital payment ecosystem. Specific objectives are:

1. Implement a device identity layer using multi-signal fingerprinting with drift-

tolerant fuzzy matching across browser and OS updates.

2. Design a composite risk scoring engine with seven independent dimensions and weights calibrated to Nepal-specific fraud patterns.
3. Achieve P95 API latency under 50 ms at $\approx 3,200$ concurrent requests per second on commodity cloud infrastructure.
4. Implement SIM swap detection via phone-to-device binding (first-write-wins) addressing the primary domestic account-takeover vector.
5. Build an async worker pipeline for behavioral profiling, device trust, and recipient risk aggregation decoupled from the hot request path.
6. Validate the system with a 168-test automated suite achieving $\geq 92\%$ pass rate.

3 Significance of the Study

Flint Secure addresses a gap in developing-market fintech: commercial solutions (Stripe Radar, Sift, Sardine) are priced for Western markets and lack Nepal-specific calibration. The project demonstrates that sub-50 ms latency, multi-dimensional scoring, device intelligence, and SIM swap detection are achievable with open-source components (Go, Redis, NATS, PostgreSQL) on Oracle Cloud Free Tier ARM64 infrastructure.

As open fintech infrastructure, Flint Secure supports Nepal’s digital financial inclusion agenda and reduces dependency on foreign fraud platforms that are economically inaccessible to local PSPs.

4 Literature Study/Review

4.1 Device Fingerprinting

Eckersley (2010) established that browser fingerprints are highly unique in large samples; Laperdrix et al. (2016) showed canvas rendering differences across GPU hardware—informing our canvas hash signal. Vastel et al. (2018) on fingerprint evolution motivated our three-layer cascade: exact hash, stored ID recovery, and fuzzy matching with evolution penalties.

4.2 Real-Time Fraud Detection

West and Bhattacharya (2016) survey ML for financial fraud; Fiore et al. (2019) address class imbalance via GAN augmentation. Dal Pozzolo et al. (2015) inform ALLOW/FLAG/BLOCK threshold calibration.

4.3 Velocity and SIM Swap

Bolton and Hand (2002) establish velocity features as highly predictive; our Redis sorted-set counters implement multi-window velocity in one pipelined round-trip. Mjaaland et al. (2021) document SIM swap in mobile banking; FIU-Nepal (2024) reports a 70% YoY increase in Nepal, justifying elevated identity-risk weighting.

4.4 *Related Systems*

Stripe Radar, Sift, and Sardine provide enterprise-scale scoring but lack Nepal-specific tuning and are cost-prohibitive at local PSP scale. Flint Secure differentiates through geographic calibration (e.g. identity weight $\times 16$ reflecting SIM swap prevalence).

5 **Proposed Methodology**

5.1 *System Architecture*

Flint Secure uses strict separation between request serving, business logic, and storage to meet a 200 ms P95 budget (targeting <50 ms achieved in testing). Each request passes middleware: request ID, GeoIP (MaxMind), API key auth (Redis cache $\rightarrow$ PostgreSQL), rate limiting, and optional HMAC-SHA256 signing with replay prevention.

The `/v1/score` handler pipelines Redis reads (device, profile, velocity), computes identity/SIM swap signals, checks quota, then invokes a pure scoring function (no I/O). Responses are sent immediately; NATS events and velocity writes run asynchronously.

5.2 *Storage*

- **Redis Persist:** devices, profiles, phone bindings, quotas (AOF, 90-day TTL).
- **Redis Cache:** velocity ZSETs, rate limits, API key cache (no persistence).
- **PostgreSQL:** history and admin data (async via workers only).
- **NATS JetStream:** `transactions.scored`, `device.first_seen`.

5.3 *Device Identification*

Three-layer cascade on `/v1/identify`: (1) exact `local_fp` hash match ($\sim 85\%$ of traffic); (2) stored device ID recovery after fingerprint drift; (3) fuzzy similarity over 14 tiered signals with block-key pre-filtering (screen, OS family).

5.4 *Risk Scoring*

Seven components (device, amount, time, velocity, network, identity, recipient) with interaction terms (e.g. new device $\times$ large amount) and hard-floor overrides (SIM swap, datacenter IP, automation signals). Identity uses SHA-256 phone hash with first-write-wins binding.

5.5 *Async Workers*

Python workers: device indexer, transaction indexer, user profile builder (percentiles, active hours), device trust, recipient risk, and config cache—keeping analytics off the latency-critical path.

5.6 *Technology Stack*

Go (chi router), two Redis instances, NATS, PostgreSQL, Python workers; SDKs for JS/Android/iOS; k6 load testing; deployment on Oracle Cloud ARM64.

5.7 UML 2.0 Design Models

The diagrams below are generated from PlantUML source files in `figures/uml/src/`. Edit the `.puml` files and run `make uml` to refresh the PDFs included here.

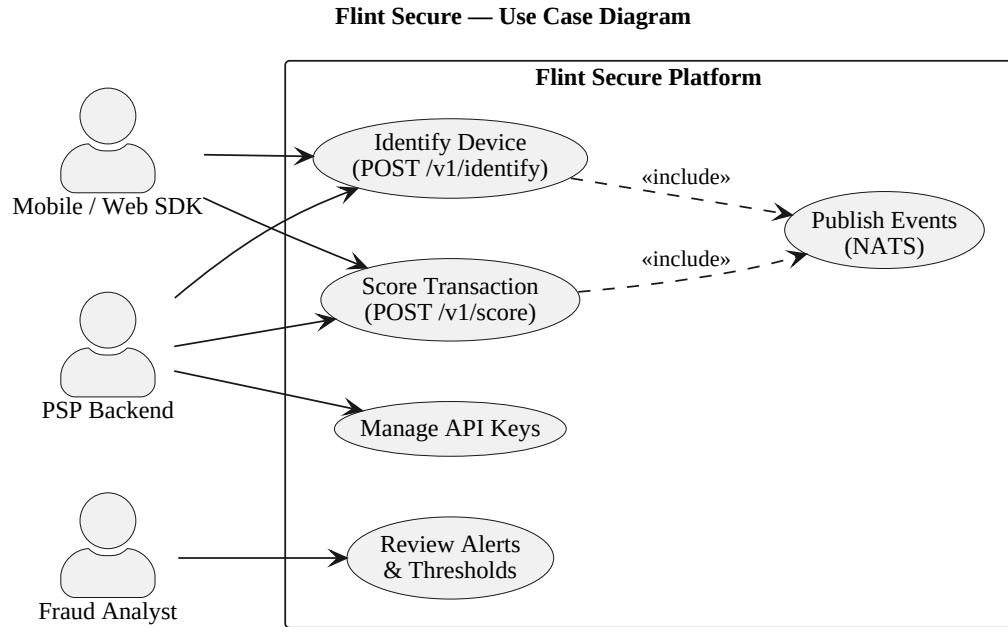


Figure 1: Use case diagram — PSP backend, mobile SDK, and fraud analyst interacting with identify, score, and administration capabilities.

6 Proposed Performance Analysis Methodology and Validation Scheme

Validation combines load testing, component tests, and acceptance criteria aligned with objectives:

Table 1: Performance targets and validation methods

Metric	Method	Target	Tool
P95 /score latency	Sustained k6 load	<50 ms	k6
Throughput	Concurrent clients	$\geq 3,200$ req/s	k6
P95 /identify L1/L3	Layer-isolated tests	<500 ms	Test suite
Error rate under load	5xx monitoring	<0.1%	k6 + logs
Functional correctness	168 automated tests	$\geq 92\%$ pass	Go/Python tests
SIM swap detection	Controlled binding tests	100% floor trigger	Unit tests

Scoring weights will be validated against FIU-Nepal fraud vectors, NRB payment statistics, and labeled pilot data (2,400 cases, Q3–Q4 2024). Latency breakdowns (Redis pipeline vs. scoring CPU) will be profiled to confirm the pure-function engine design.

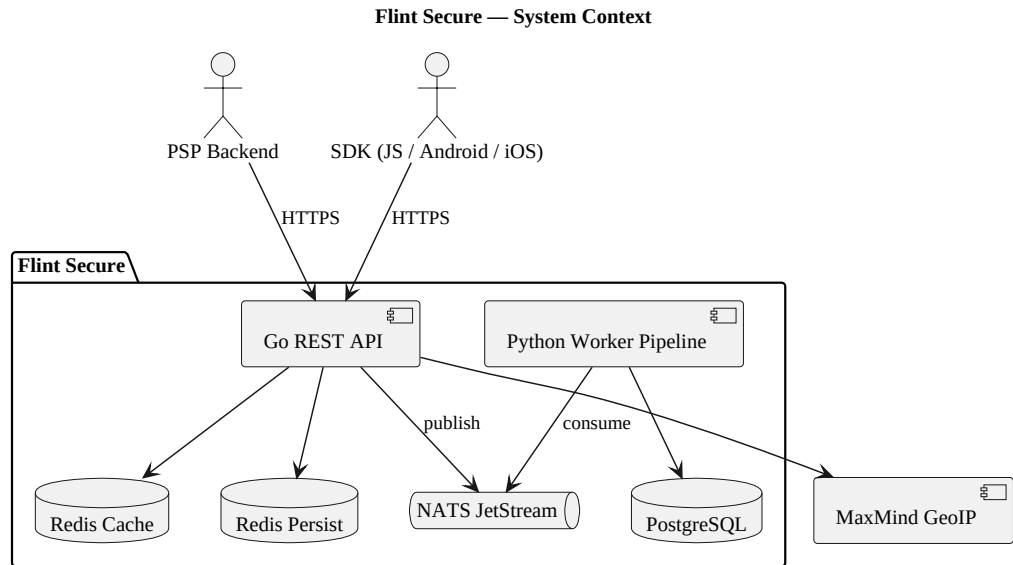


Figure 2: System context diagram — Flint Secure boundary with Go API, dual Redis, PostgreSQL, NATS workers, and MaxMind GeoIP.

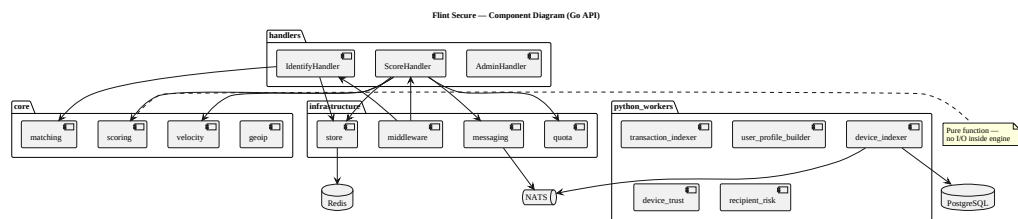


Figure 3: Component diagram — Go API packages (handlers, scoring, matching, velocity, middleware) and Python workers.

7 Proposed Deliverable/Output

8 Project Task and Time Schedule

9 Bibliography/References

References

- [1] R. J. Bolton and D. J. Hand, Statistical fraud detection: A review, *Statistical Science*, 17(3):235–255, 2002.
- [2] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, Calibrating probability with undersampling for unbalanced classification, *IEEE Symposium Series on Computational Intelligence*, 2015.
- [3] P. Eckersley, How unique is your web browser? *Privacy Enhancing Technologies Symposium (PETS)*, 2010.
- [4] U. Fiore et al., Using generative adversarial networks for improving classification effectiveness in credit card fraud detection, *Information Sciences*, 479:448–455, 2019.

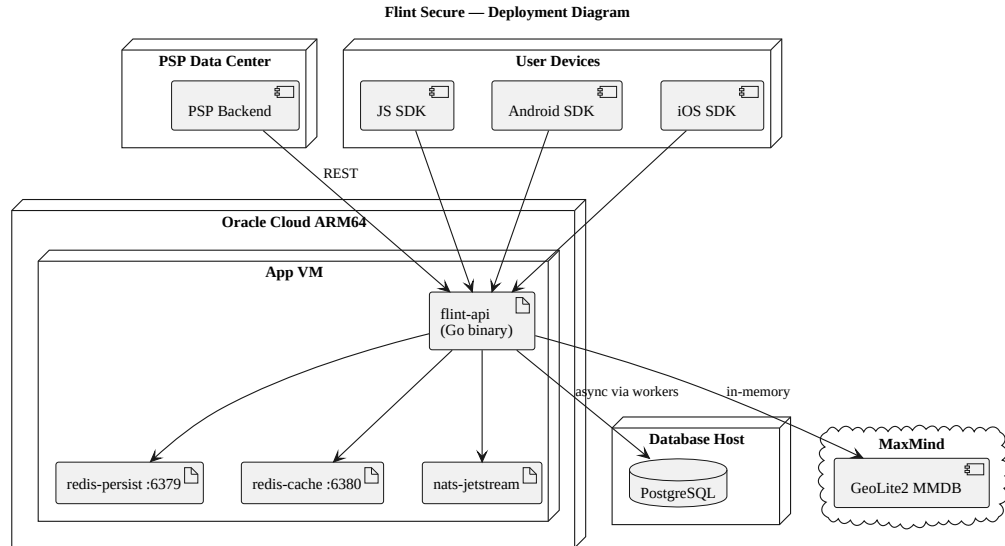


Figure 4: Deployment diagram — Oracle Cloud ARM64 application VM, PostgreSQL host, PSP backend, and device SDKs.

Table 2: Major deliverables

Deliverable	Description	Phase
REST APIs	/v1/identify, /v1/score with auth and signing	Wk 6
Device SDKs	JavaScript, Android, iOS signal collectors	Wk 8
Scoring engine	Seven-component rules + interaction terms	Wk 9
Worker pipeline	NATS consumers for profile, trust, risk	Wk 10
Data stores	Redis schemas, PostgreSQL migrations	Wk 7
Test suite	168 tests (identify, score, auth, quota)	Wk 12
Documentation	API reference, deployment guide, proposal/report	Wk 14

- [5] Financial Intelligence Unit Nepal (FIU-Nepal), *Annual Report 2023/24*, Nepal Rastra Bank, 2024.
- [6] P. Laperdrix, W. Rudametkin, and B. Baudry, Beauty and the beast: Diverting modern web browsers to build unique browser fingerprints, *IEEE Symposium on Security and Privacy*, 2016.
- [7] B. B. Mjaaland, J. Heeger, and S. D. Wolthusen, SIM swap: A new insider attack in the mobile banking ecosystem, *IEEE CSR*, 2021.
- [8] A. Vastel et al., FP-STALKER: Tracking browser fingerprint evolutions, *IEEE Symposium on Security and Privacy*, 2018.
- [9] J. West and M. Bhattacharya, Intelligent financial fraud detection: A comprehensive review, *Computers & Security*, 57:47–66, 2016.

Flint Secure — Core Domain Classes

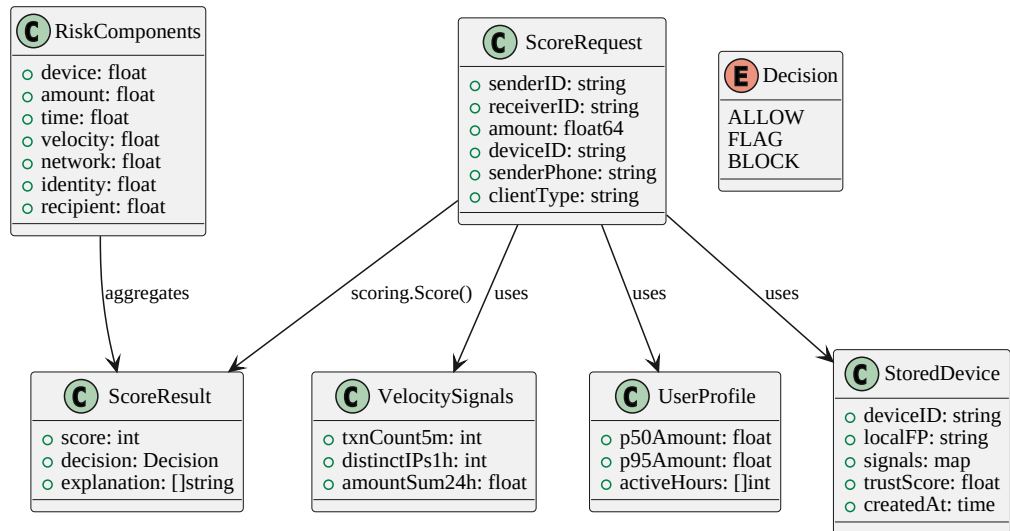


Figure 5: Class diagram — core domain types including `StoredDevice`, `ScoreRequest`, `ScoreResult`, and risk components.

Table 3: Semester task schedule (14 weeks)

Wk	Phase	Duration	Activities
1–2	Planning	2 wks	Requirements, architecture, literature finalization
3–4	Foundation	2 wks	Go project skeleton, Redis/Postgres, middleware chain
5–6	Identify API	2 wks	Fingerprint cascade, block keys, SDK integration
7–8	Score API	2 wks	Velocity, scoring engine, SIM swap, quota
9–10	Async layer	2 wks	NATS, Python workers, profile/trust builders
11–12	Validation	2 wks	k6 load tests, 168-test suite, bug fixes
13–14	Defense prep	2 wks	Proposal/mid-term/final report, demo, slides

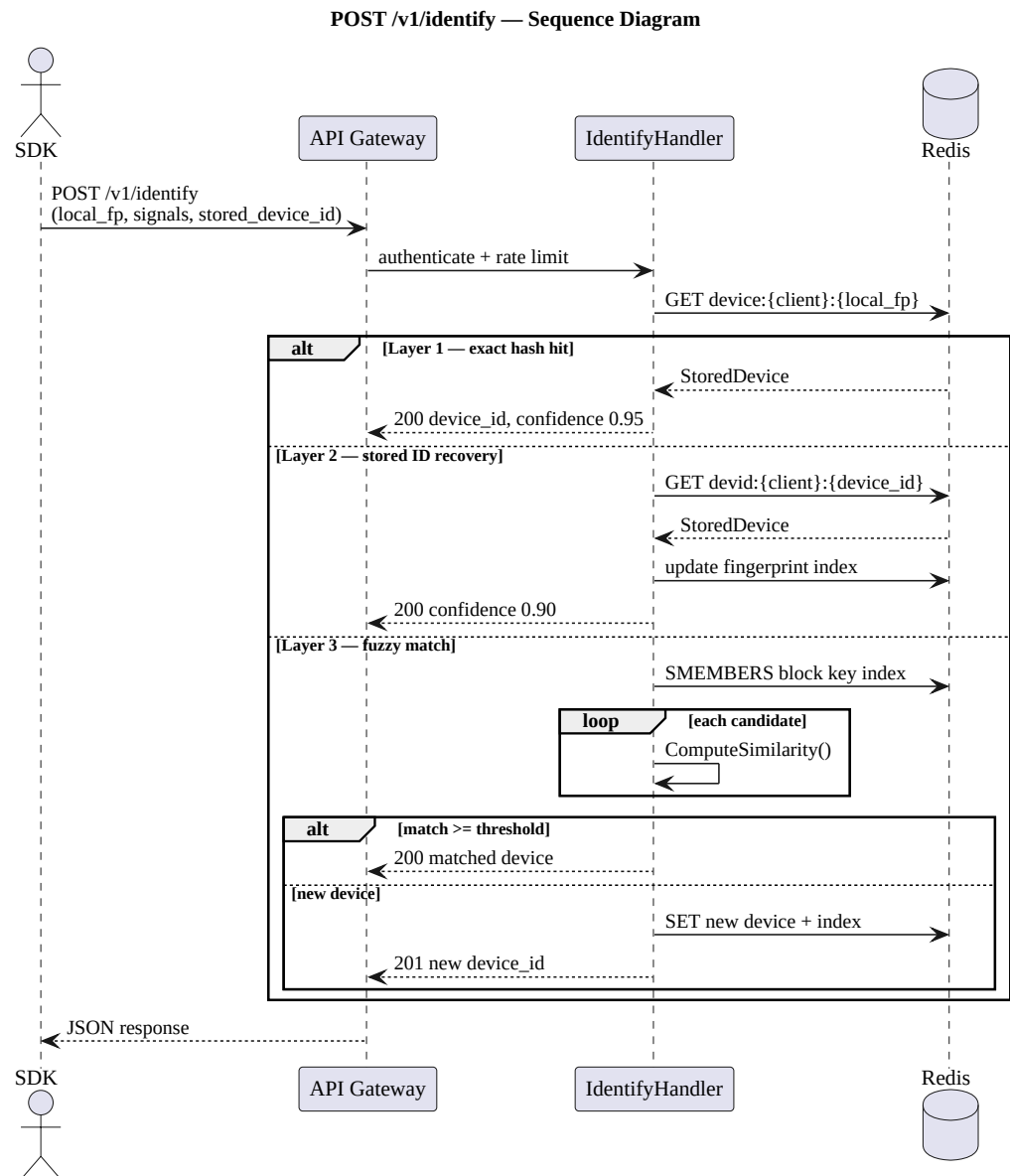


Figure 6: Sequence diagram — POST /v1/identify three-layer device matching cascade.

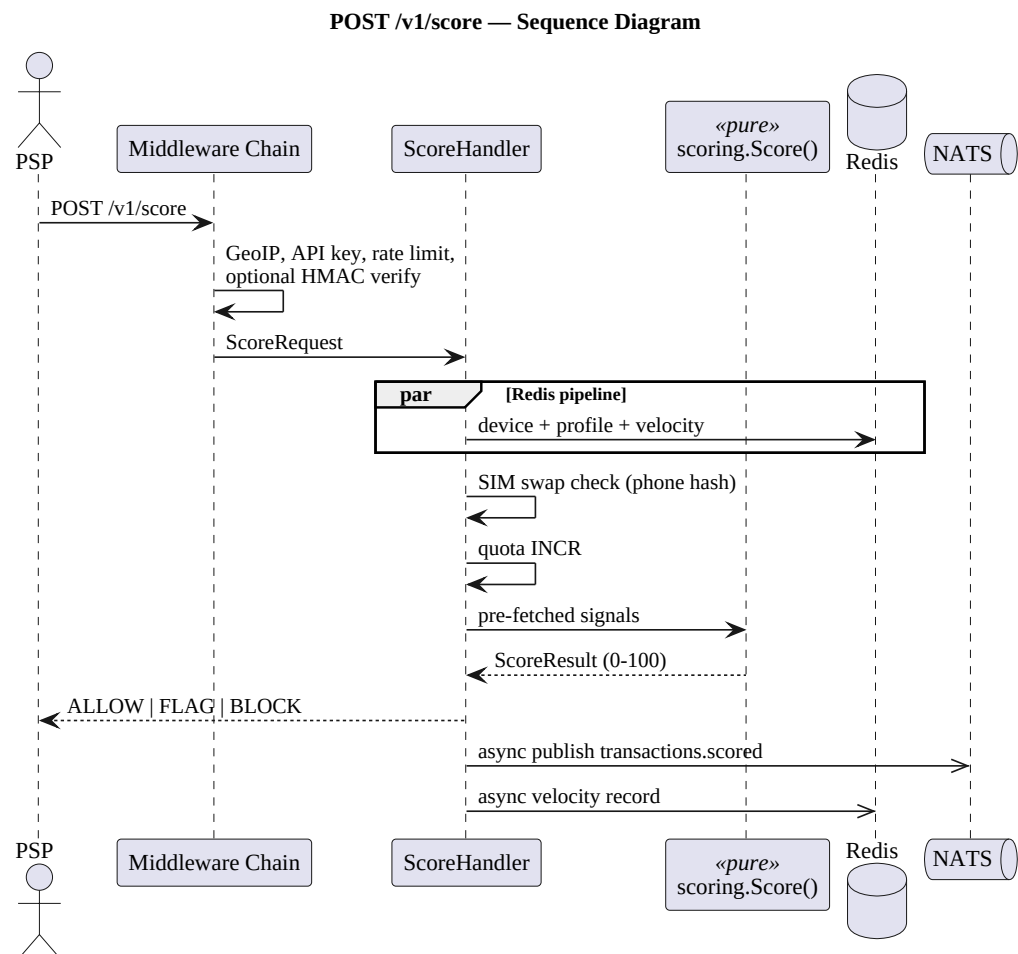
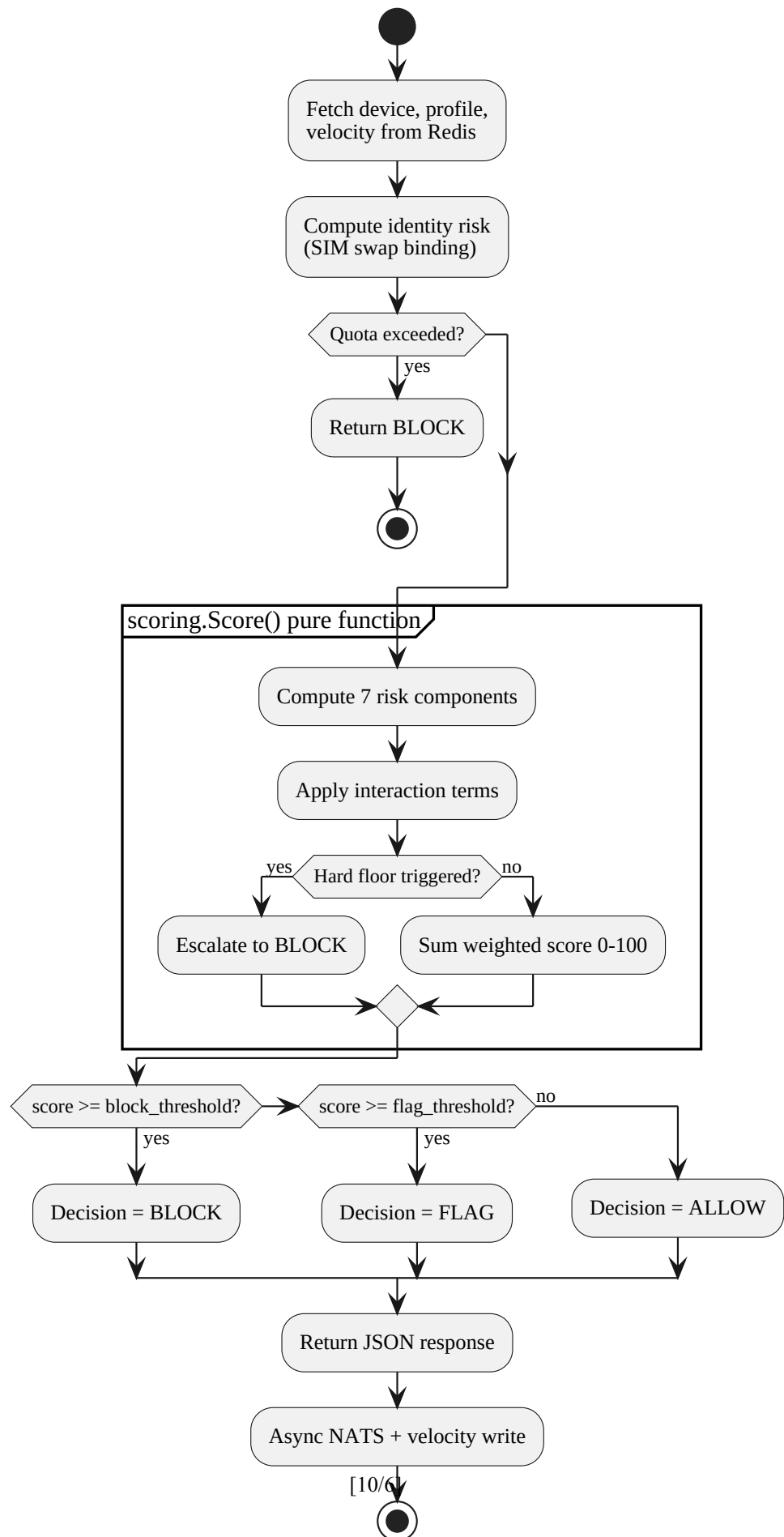


Figure 7: Sequence diagram — `POST /v1/score` middleware chain, Redis pipeline, pure scoring, and async NATS publish.

Risk Scoring — Activity Diagram**Figure 8:** Activity diagram — risk scoring from signal fetch through ALLOW, FLAG, or